

マルチエージェント・コード生成における セッション継続の費用

商用コーディングエージェント CLI の挙動測定と事前指定比較

The Economics of Session Affinity in Multi-Agent Code Generation:
A Measurement Study of a Commercial Coding-Agent CLI

Fudo

2026 年 9 月 5 日 統合改訂稿 v0.3

要旨

関連するコーディング作業を同じ会話で続ければ背景の再取得を減らせるが、履歴の処理費用と実行順の制約も生じる。本研究は商用 CLI のキャッシュ挙動を測定し、費用モデルと編集範囲の重なりによる会話継続方式を評価する。1 つの Python リポジトリ内の 7 仕様について、新規方式 B と継続方式 C を 98 対・196 ラン比較した。相対費用差中央値は +5.51%、BCa 95% 区間は $[-3.18, +10.12]\%$ で、事前指定した 20% 削減の達成条件は満たされなかった。時間と受入全通過率は、15% と 10 ポイントの許容差に対する非劣性基準を満たした。事後分析では、継続が発生しない W 形状 2 仕様の中央値も -7.9% と $+12.1\%$ に分かれ、全体の小さな正の差を継続一般の効果とする解釈に注意を促した。CT 仕様は全 14 対で高く、中央値 +50.6% だった。CT 以外では C が安い対が 44/84 で、全体平均の増加は CT の大きな差に集中していた。CT の総ターン数は 587 から 572 へ減る一方、cache read は 32.6%、cache write は 96.8% 増えた。タスク別には、継続した 2 タスクがターン数を 2~3 削減しながら最大文脈量を 46% と 91% 増やし、継続しない先頭タスクも 14 反復すべてで費用が増えた（中央値 +42.8%）。また補助モデルの使用記録は、新規セッション 434 件と継続セッション 84 件で完全に分かれた。固定単価での換算増分は cache write が最大項目だったが、CLI 報告額との残差があり、請求内訳や継承履歴だけの費用は同定できなかった。継続対象 5 仕様のモデル予測は 4 件が誤差 25% 以内で、5 件とも過大予測だった。本研究は陰性対照、仕様別の異質性、残差を保持した費用分解を示す。CT は 1 仕様に限られ、継続と実行順制約の因果効果は分離できない。

キーワード：コーディングエージェント、セッション継続、プロンプトキャッシュ、費用計測、事前指定分析

Abstract

Continuing related coding tasks in one session can reduce repeated exploration while carrying history and adding scheduling constraints. We characterize a commercial coding-agent CLI and compare fresh sessions (B) with write-scope-connected lane continuation (C) on seven specifications in one Python repository (196 runs; 98 pairs). The median paired relative cost difference is +5.51% (BCa 95% interval: $[-3.18, +10.12]\%$), failing the pre-specified 20% reduction criterion. Runtime and acceptance meet non-inferiority criteria with margins of 15% and 10 percentage points, respectively. Post-hoc analysis identifies two structural controls with no continuation: their medians are -7.9% and $+12.1\%$. The CT specification is costlier in all 14 pairs, with a median difference of $+50.6\%$; outside CT, C is cheaper in 44/84 pairs. The aggregate mean increase is concentrated in CT. CT's reported turn count falls from 587 to 572, while cache-read and cache-write tokens rise by 32.6% and 96.8%. Per task, the two continued tasks take 21–30% fewer turns while their peak context grows by 46% and 91%, and the leading task—fresh in both arms—is costlier in all 14 replications (median $+42.8\%$). Auxiliary-model use separates completely: it is recorded in all 434 fresh calls and in none of the 84 resumed ones. Fixed-rate accounting assigns the largest standardized increase to cache writes, but a residual relative to CLI-reported cost prevents a complete billing decomposition or attribution to inherited history alone. Excluding two structural unity predictions, four of five model predictions are within 25% relative error and all five overpredict. The study combines structural controls, workload heterogeneity, and residual-preserving cost accounting. A single CT specification and a bundled intervention limit causal interpretation and generalization.

版と証拠の位置づけ。本稿は 2026 年 9 月 5 日の統合改訂稿 v0.3 である。v0.2 で行った H1/H3 の実装訂正と旧結果は付録Aに集約した。v0.3 の陰性対照、符号検定、継続対象のみの予測評価、費用分解は結果を見た後の追加分析である。事前指定の主要判定を置き換えず、旧稿・生データ・固定予測を保持する。独立した公開登録時刻は確認できていないため「事前指定」と表記する。分析手順と再現物を付録C、Dに示す。

目次

1 導入	4
2 背景と関連研究	4
2.1 商用 API のキャッシュと文脈再利用	4
2.2 複数エージェントの作業分割	5
2.3 キャッシュ親和性と負荷分散：操作する層の違い	5
2.4 実務の問題報告と計測ツール	5
2.5 事前指定と本稿の位置	5
3 キャッシュ挙動の測定と費用モデル	6
3.1 測定単位と用語	6
3.2 観測したことと、その限界	6
3.3 説明用の分解と損益分岐	7
3.4 固定した予測の実際の計算法	7
4 比較する実行方式	8

4.1	共通の制約とレーンの定義	8
4.2	直接の競合と、推移的な直列化	8
4.3	失敗・切替・中断の扱い	8
5	実装・実験方法	9
5.1	対象・仕様・規模	9
5.2	順序・時間間隔・キャッシュ共有の抑制	9
5.3	記録と費用・品質の定義	10
5.4	中断と停止条件	10
6	評価と結果	10
6.1	評価対象と事前指定の条件	10
6.2	補助分析の位置づけ	11
6.3	費用：20% 削減の達成条件は満たされなかった	11
6.4	仕様別と CT 除外の感度分析	11
6.5	構造的陰性対照から見た変動	12
6.6	CT の料金項目とタスク別の費用分解	13
6.7	継続に伴う機構の観測	15
6.8	時間と受入テスト	16
6.9	モデル予測の一致と不一致	16
6.10	中心主張と追加分析	17
7	支持されなかった見立て	18
7.1	キャッシュ共有を単純な二択で捉えない	18
7.2	履歴の増大と書き直しの関係	18
7.3	予算付きレーンを成果としない	18
8	妥当性と適用範囲の限界	18
8.1	対象の狭さと仕様への依存	18
8.2	複合介入と観測できない状態	18
8.3	順序・運用変更と統計上の仮定	19
8.4	モデル近似と校正の量	19
8.5	品質の範囲と少数の不一致	19
8.6	装置を再配布しないことによる再現の非対称	19
8.7	非盲検と公開前の証拠	19
9	結論	20
A	分析訂正の履歴	20
B	機構測定記録と未確定事項	21
C	追加分析の定義	21
C.1	v0.3 の追加分析と計装の照合	21
D	再現方法と成果物	22

1 導入

複数の AI にコード作成を分担させると、個々の AI が同じリポジトリのファイル構成、関数、実装上の約束を調べ直すことがある。人間の仕事にたとえば、新しい担当者にも何度も背景を説明する状況に近い。同じ担当者に続けて仕事を頼めば説明を省けそうだが、言語モデルでは過去の会話やツール出力も次の入力に含まれ、その処理に費用がかかる。キャッシュはこの入力の単価を下げて、処理対象の文脈量をゼロにはしない。

本研究の問いは「会話が継続できるか」ではなく、「継続による再取得の節約が、履歴を引き継ぐ費用を上回るか」である。対象は推論サーバや KV キャッシュを変更できない商用コーディングエージェント CLI であり、利用者が選べる実行順とセッション継続を調べる。KV キャッシュとは、モデルが入力を処理した中間状態を再利用するための仕組みである。本稿が直接取得するのはその内部状態ではなく、CLI が報告する読み取り・作成トークン数と費用である。

既存研究は、キャッシュ方式の費用評価 [7]、文脈の保存と縮約 [11]、タスク分割による時間・品質・費用の改善 [14] を扱っている。本稿はこれらの先行成果を前提とし、特定 CLI の継続境界の測定、実測に基づくモデルの検証、編集対象の重なりを利用する 1 つの継続方式の比較を組み合わせる。キャッシュ計測そのものや、関連タスクをまとめる発想が新しいとは主張しない。

貢献は次の 3 点である。

1. 継続時の履歴読み、新規セッション間の共有の条件、固定文脈、初回書き直しについて、クライアント記録から分かる範囲と分からない範囲を整理する。
2. 再取得の節約と履歴の運搬費用を分解し、独立した B 側の較正データで固定した予測を本実験に照合する。
3. 新規方式 B とレーン継続方式 C を比較し、構造的陰性対照で見える変動、CT 仕様に集中する増加、料金項目・タスク別の費用分解を示す。事後分析は事前指定の比較と区別する。

適応スケジューラや会話の最適な切り替え規則は開発・評価していない。観測結果を根拠に改善候補を挙げることに、その改善を実証することを区別する。

2 背景と関連研究

2.1 商用 API のキャッシュと文脈再利用

Lumer ら [7] は、商用 3 社の API について、全履歴、システムプロンプトのみ、動的なツール結果を除く方式を比較し、DeepResearch Bench 上で費用と最初のトークンまでの時間を評価している。本稿との差は、キャッシュの有無の一般的評価ではなく、コーディング CLI の新規／継続境界と、複数タスクをまとめる実行方式を対象とする点にある。

Santoni の Contextual Memory Virtualisation (CMV) [11] は、会話状態を有向非巡回グラフで管理し、保存、分岐、縮約を扱う。76 のコーディングセッションによる事例評価を含み、再構築する文脈の費用も論じる。その観察は本稿の動機に関連するが、CMV の縮約・分岐と本稿のレーン継続は異なる介入である。本稿の B/C 比較を、CMV への直接の反証として解釈しない。

2.2 複数エージェントの作業分割

Co-Coder[14] は、依存グラフの分割とスケジューリングにより、エージェント間の文脈転送と並列性の関係を扱う。28 課題で通過率、時間、API 費用を評価し、費用削減も報告している。したがって「既存の協調研究は費用を測っていない」という差別化は成り立たない。本稿ではタスク分割方式を最適化せず、既存タスクの宣言した編集範囲からレーンを構成し、その継続方式と CLI の挙動を測る。

2.3 キャッシュ親和性と負荷分散：操作する層の違い

SGLang[16] は RadixAttention による KV 再利用を実行基盤に組み込み、Parrot[5] は要求間の関係をサービス側へ伝える。その上で、どの実行器へ要求を送るかという選択には、キャッシュの局所性と負荷分散の競合がある。DualMap[15] は prefix に基づく 2 候補への写像と負荷に応じた選択を組み合わせ、この競合を扱う。実装面でも Ray の PrefixCacheAffinityRouter[10] は負荷不均衡時に分散を優先し、llm-d[6] は KV イベントに基づく prefix 局所性のスコアと負荷スコアを組み合わせる。

AgentServeSim[9] の §6.1 は、同じ program の後続ターンを同じ推論器へ固定する方式をシミュレーションで比較した。対象の 2 インスタンス構成では、固定ルーティングが round-robin や least-loaded より高い hit 率と短い p95 完了時間を得た。これは親和性が有利となる関連所見だが、本稿とは介入も評価量も異なる。同じ会話内容を別の推論器に送る比較と、複数の開発タスクを同じ会話へ統合して履歴・実行可能順を変える比較は同じではない。前者は再計算を避ける利得を、後者は文脈量の増減を含む利用者の費用を評価する。したがってサービング層での時間短縮と、本稿の CT における USD 費用増加は両立し、逆符号の同一効果を示すものではない。

2.4 実務の問題報告と計測ツール

Claude Code の issue 42338[12] では、利用者が再開直後の cache creation 増加をトークン記録付きで報告している。また ccusage[2] はセッションごとの入出力・cache write/read・費用を可視化し、保存された費用値と料金表による再計算も区別する [1]。これらはセッション再開の費用が実務上の関心であり、観測手段が既に存在することを示す。特定環境の問題報告から「古いセッションが常に最も高い」「広く信じられている」とは結論しない。本稿はそうした問題を動機に、固定したタスク仕様と比較方式の下で、受入結果を含めた費用を測定する。

2.5 事前指定と本稿の位置

エージェント実験ではモデル、プロンプト、評価指標、結果を見た後の変更などの選択肢が多く、事前登録の必要性が論じられている [13]。本研究は判定規則と変更履歴を実験前から保存した。ただし公開された独立時刻証明は確認できず、実験中の運用変更と、分析後の実装訂正もある。そのため、事前に指定した部分と後から追加・訂正した部分を明示する。本稿の貢献は測定と比較の記録にあり、新しい最適化問題の計算量や最適解法を提示するものではない。

3 キャッシュ挙動の測定と費用モデル

3.1 測定単位と用語

セッションは、履歴を引き継いで再開できる会話の単位である。タスクは 1 件の開発作業、ランは 1 仕様を 1 方式で実行して受入判定する単位とする。1 タスクの CLI 呼び出しの内部で、モデルとツールの複数回のやり取り（ターン）が発生しうる。短い挙動確認をプローブと呼び、本実験の 196 ランとは区別する。

キャッシュに以前の入力が利用できる状態を **warm**、その読み取りが失われる状態を **cold** と呼ぶ。ただし、固定文脈だけが残る中間状態もある。読み取り比・作成比で機械判定した **ambiguous** を、都合よく **warm** や **cold** へ読み替えない。プローブは主に CLI 2.1.220、本実験は 2.1.247 であり、プローブの挙動が本実験の全呼び出しで同一だったとは仮定しない。

3.2 観測したことと、その限界

測定	観測	解釈の範囲
M1：warm 再開	一意な入力接頭辞で 3 反復。履歴読み比は全て 1.0000	小さな追加要求で履歴のキャッシュ読みを確認。仕事全体の費用削減を意味しない。
M4：新規から新規	3 反復で入力の cache write/read 内訳が一致	この条件では共通ユーザー入力による追加割引を確認しなかった。全条件での非共有は主張しない。
固定文脈・交差共有	新規要求でも約 21.6k の cache read。継続後の別の新規要求で共有を観測	固定文脈の共有と、ユーザー履歴の条件付き共有を分ける。
M6：履歴増加	2 系列。後続の cache write は新規追加分でほぼ一定	継承履歴が増えると毎回の書き込みも必ず増える、という見立ては支持されない。
M7：追加量	500-16,000 文字の掃引と 2,000 文字の 6 反復	書き直しは追加量の単調な閾値では説明できない。初期の cache read との関連は観測の所見。
M2・M3：間隔／切替	長間隔 2 件、モデル切替 2 件の分類は ambiguous	正確な寿命や「モデル切替で全損」を確立していない。

表 1: 機構測定の要約。詳細ログと判定単位は付録Bに示す。k は千トークン。

M1 の 3 反復では、継承履歴 45,124、45,123、45,122 トークンが、それぞれ同数の cache read として報告された。再開呼び出しの費用は 0.01438 USD で、直前の初期呼び出しとの費用比は中央値で約 1/11.1 だった。これは短いプローブ内の呼び出し比較であり、同じコーディング課題を解く B/C の比較ではない。

M7 では 500 文字の追加で書き直しが起き、16,000 文字では起きない例を得た。初期

cache read が少ない観測と書き直しが対応したが、提供側のキャッシュ状態を独立に操作していない。したがって「状態が原因だと特定した」ではなく、「追加量だけでは説明できず、初期状態の指標と対応した」と述べる。事前の確実な制御は未確立でも、利用者側に全く情報がないわけではない。事後の観測に基づく適応は候補として残る。

3.3 説明用の分解と損益分岐

タスク j を新しく始めるときの再取得費用を e_j 、前タスク i から引き継ぐ履歴を H_i 、継続後のターン数を K'_j とする。基準入力単価を p_{in} 、キャッシュ読み・書きの単価係数を α_r, α_w とし、初回に履歴を書き直す指示変数を $\rho \in \{0, 1\}$ と置く。履歴の運搬費用は、単純化したモデルでは

$$c_{\text{carry}}(i, j) \simeq p_{\text{in}} \{ \rho \alpha_w + (K'_j - \rho) \alpha_r \} H_i \quad (1)$$

となる。本来の作業と固定文脈の費用が方式間で等しいと近似できるなら、継続が得になる条件は

$$c_{\text{carry}}(i, j) < e_j \iff H_i < H_j^* = \frac{e_j}{p_{\text{in}} \{ \rho \alpha_w + (K'_j - \rho) \alpha_r \}}. \quad (2)$$

e_j は USD、 H_i はトークン、 p_{in} は USD/トークンである。キャッシュが使えても、長い履歴を多数のターンで処理すれば費用が増えることを表す。

この式は意思決定の完成形ではない。継続はターン数、出力、再探索の順序も変えうるため、共通費用の相殺は近似にすぎない。また書き直しの指示変数を、追加プロンプトの大きさや公称保持時間だけで確実に決めることはできない。再開間隔は利用者が動かせても、キャッシュ全体の状態は直接制御できない。

3.4 固定した予測の実際の計算法

モデル検証には、アーム B だけを実行した 7 仕様各 1 反復の較正データを用いた。編集系ツール (Write、Edit、NotebookEdit、MultiEdit) が最初に現れる前を探索区間、以後を作業区間と定義する。各タスクの最小文脈量を $\hat{S}_j$ とし、探索ターン t でそれを超える文脈量の和を

$$\hat{E}_j = \sum_{t \in \mathcal{E}_j} \max(0, \text{context}_{jt} - \hat{S}_j) \quad (3)$$

とする。これは探索の意味を直接測った量ではなく、ツール名とトークン数による代理量である。

固定した実装 `analysis/calibrate.py` は、B 側の実測費用を土台とし、レーンの 2 件目以降を

$$\hat{K}'_j = \max(1, K_j - K_{E,j}), \quad (4)$$

$$\hat{C}_{C,j} = C_{B,j} - p_{\text{in}} \alpha_r \hat{E}_j + p_{\text{in}} \{ \rho \alpha_w + (\hat{K}'_j - \rho) \alpha_r \} \hat{H}_i \quad (5)$$

で予測する。ここで $\rho = 1$ に固定し、 $\hat{H}_i$ は直前タスクの B 側較正での最大文脈量を使う。レーン先頭は $\hat{C}_{C,j} = C_{B,j}$ で、仕様内を合算して $\hat{r}_s = \sum_j \hat{C}_{C,j} / \sum_j C_{B,j}$ を得る。説明用の式と実装の近似を区別し、実装が継続による累積履歴を逐次シミュレーションしているとは書かない。

較正実装の単価は、記録された `claude-sonnet-5` に対し 100 万トークン当たり入力 3.00、出力 15.00、cache read 0.30、cache write 3.75 USD である。これは当該予測コードの固定数で、現在のサービス価格の案内ではない。出力費用を含む B 側実測費用を基礎にしているが、継続による出力の変化や別のキャッシュ作成料金は明示的に予測しない。予測ファイルは本実験前のコミット `4fbb713` に保存されており、本稿の訂正で再較正していない。

4 比較する実行方式

4.1 共通の制約とレーンの定義

タスク間の先行関係を守り、宣言された編集対象が直接重なるタスクを同時に実行しない。方式 B はこの実行中タスクとの直接の重なりを確認し、各タスクを新規セッションで呼ぶ。方式 C は同じ共通制約に加え、編集範囲が重なるタスクを辺で結んだ無向グラフの連結成分をレーンとする。連結成分とは、直接または他のタスクを経由してつながったまとまりである。同一レーンは同時に 1 タスクだけとし、2 件目以降を前のセッション ID で再開する。別レーン間の並列性は残す。

4.2 直接の競合と、推移的な直列化

依存関係がない 3 タスク A, B, C について、編集範囲 W_A, W_B, W_C が

$$W_A \cap W_B \neq \emptyset, \quad W_B \cap W_C \neq \emptyset, \quad W_A \cap W_C = \emptyset \quad (6)$$

なら、B 方式は A と C を同時に実行できる。一方、C 方式では 3 タスクが同じ連結成分なので全て順番に実行する（図1）。「直接重なるタスクは同時に走れない」ことから「連結成分内は全部順番にしても追加の代償がない」とは導けない。

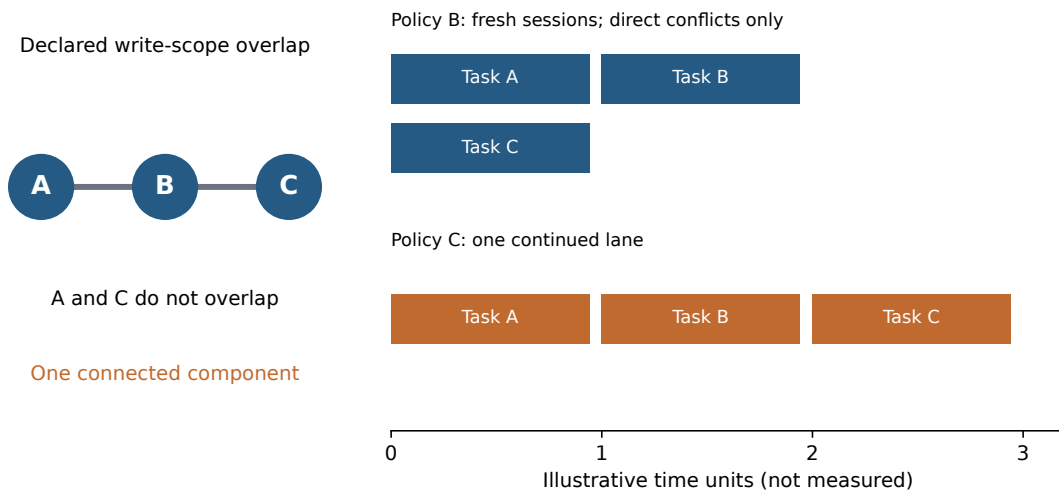


図 1: CT の構造を示す模式図。タスク A-B、B-C に編集範囲の重なりがあるが、A-C にはない。棒の長さは模式的で、実測の所要時間ではない。タスク名 A/B/C と方式名 B/C を区別する。

この比較は会話継続とレーン内直列化を同時に変える複合的な介入である。したがって方式 C の費用差を、直列化だけ、あるいはキャッシュだけの因果効果として分離できない。同じスケジュールで新規／継続だけを変える追加アームは、本研究では評価していない。

4.3 失敗・切替・中断の扱い

代表モデルの不一致時にはセッションの継承をやめる処理がある。本実験は代表モデルを固定したが、CLI 内部の補助モデルの使用までゼロに固定してはいない。一方、実装を照合すると、タスク失敗を判定する前にも取得済みセッション ID が記録される。そのため「失敗時には必ずセッションを破棄する」という旧稿の説明は採用しない。先行タスク失敗時の依存先ブロッ

クは行うが、失敗文脈からの継続を全経路で防ぐ保証は評価していない。ラン全体の受入不合格と、途中のタスク例外も区別する。レーン長の上限、適応的な分割、長い中断後のキャッシュ保持保証は実装・評価の成果として主張しない。

5 実装・実験方法

5.1 対象・仕様・規模

実行基盤は既存のマルチエージェント・フレームワーク、実験対象は Python の `square-media` リポジトリである。7つの独立したリポジトリを比較したわけではない。対象を、受入テストがあり実装が未完成な3つのコミット（`d4449a0`、`3be3676`、`6f5b695`）に固定した。各ランは別の `Git worktree` で実行し、他ランの生成物を引き継がない。

仕様	形状	レーン内のタスク数	役割
<code>w_two_files</code>	W	1 + 1	継続対象がない対照
<code>n_db_chain</code>	N	2	依存関係ですでに直列
<code>m_mixed</code>	M	2 + 1 + 1	依存鎖と独立作業の混合
<code>s17_w_two_files</code>	W	1 + 1	別系列の対照
<code>s17_n_db_chain</code>	N	2	別系列の依存鎖
<code>s17_m_mixed</code>	M	2 + 1	別系列の混合
<code>ct_library</code>	CT	3	依存なし・編集範囲の推移的な重なり

表 2: 7 仕様の構成。W の差はセッション再利用の効果ではなく、独立に実行したランの変動も含む。

各仕様を 2 方式で 14 反復し、98 対・196 ランを取得した。反復数はパイロットと事前指定の手順に基づく。本実験の期間は 2026 年 8 月 28 日から 9 月 2 日で、記録されたモデル ID は `claude-sonnet-5`、CLI は 2.1.247 である。モデル名は当該ログの識別子として記載する。最終データセットに欠測対・事後の外れ値除外はないが、その完成までに中断・再実行は発生した。

5.2 順序・時間間隔・キャッシュ共有の抑制

`build_schedule` は乱数種 20260801 で仕様順を並べ替え、奇数反復を B 先行、偶数反復を C 先行にする。アーム順そのものを各対で無作為抽出する設計ではない。保存された終了時刻から `wall_s` を引いて開始時刻を復元すると、B 先行 49 対、C 先行 49 対で、全 98 対が計画の先行方式と一致した。

実験途中に、全仕様の片方式を先に実行する順序から、仕様ごとに両方式を隣接して実行する順序へ変更した（A-5）。利用上限で比較の片側だけが残る状況への対応であり、費用判定の閾値を変更したものではない。実際の先行ラン終了から後続ラン開始までの間隔は中央値 0.27 秒、最大 38.24 分で、10 分以上離れた対は 11 対だった。終了時刻と丸めた所要時間からの推定なので、サブ秒値は精密な待機時間の測定ではない。

各ランのプロンプトに固有文字列（`nonce`）を置き、別ランのユーザー入力接頭辞が一致しにくいようにした。同一ラン内ではその文字列を維持する。この措置は固定システム文脈の共有やサービス側の時間変動を除去しない。順序の均衡だけでキャッシュ状態の交絡がなくなったとはせず、離隔対を除く感度分析も報告する。

5.3 記録と費用・品質の定義

CLI 呼び出しごとの `calls.jsonl` に、タスク ID、セッション ID、継続の有無、モデル、CLI 版、入出力とキャッシュのトークン数、費用、ターン数を保存した。--output-format stream-json --verbose から取得したターン情報も残す。総費用は提供側が報告した USD 費用をラン内で合計した値であり、請求書との独立照合はしていない。データセットと `run.json` の費用は全 196 ランで一致した。会計分析はこのラン集計を使い、追記された呼び出し履歴の全行合計とは区別する。CLI の代表モデル ID は費用が最大のモデルを表し、補助モデルの使用もある。ログには `models_used` の名前を保存したが、元の `modelUsage` のモデル別金額・トークン内訳は残していない。

時間はランの投入から受入判定までの経過時間であり、モデルの推論時間だけではない。品質は受入テストが全て通ったランを 1、それ以外を 0 とする二値である。個々のテストの平均通過率、コードの保守性、人間による品質採点はこの指標に含まれない。宣言した編集範囲と実際の書き込みの一致率、範囲逸脱、LLM 審判による採点は測定していない。

5.4 中断と停止条件

旧集計はインフラ中断イベント 121 件を 196 ランで割り、61.7% として 20% の停止条件を検出した。A-6 は、繰り返し中断された同一ランをまとめると 27/196 (13.8%) であると記録し、比較値の判定前に再開を決めた。本稿は A-6 の根拠と、旧集計が停止を検出した事実の両方を残す。イベント比率とラン比率は異なる指標であり、どちらかを無説明で消さない。利用上限、ローカル遮断の誤検知、worktree 不整合、実行順変更などの運用上の介入があり、その個別の影響を分離評価していない。

6 評価と結果

6.1 評価対象と事前指定の条件

対 i の相対費用差を $d_i = (C_i - B_i)/B_i$ とする。負なら C が安い。主要な点推定は全 98 対の $\text{median}(d_i)$ であり、仕様別中央値の中央値ではない。同数反復の 7 仕様を含む固定構成の集約値として解釈する。

仮説	問い・推定対象	事前指定の達成条件
H1 費用	対ごとの相対差、中央値	両側符号反転検定 $p < 0.05$ 、BCa 95% 区間が 0 を跨がない、かつ中央値 $\leq -20\%$
H2 時間	$\text{median}(T_C)/\text{median}(T_B)$	片側 BCa 上限 < 1.15 。実装は 90% を採用
H3 受入	$p_C - p_B$	片側 90% 下限 > -10 ポイント
H4b 予測	仕様別の総費用比 r_s	予測との相対誤差 25% 以内が過半数、かつ全仕様で優劣の方向一致

表 3: 判定の要約。H1 の相対尺度は分析前文書の決定 A に基づく。H2/H3 の許容差は運用上の判断値である。

符号反転検定の統計量は平均、費用の実質性と区間の対象は中央値である。再標本数 10,000、乱数種 20260802 とし、検定の p 値は $(b+1)/(10,000+1)$ で計算する [8]。BCa はバイアス

補正と加速度を用いるブートストラップ区間である [4]。検定と区間が異なる統計量を扱うため、 $p < 0.05$ でも中央値の区間が 0 を跨ぐことは矛盾ではない。独立性と符号交換可能性を仮定した分析であり、決定的な交互順序から無作為化推論が自動的に成立するとはしない。

6.2 補助分析の位置づけ

本節の陰性対照、正確符号検定、継続対象 5 仕様のモデル評価、CT 費用分解、および §6.7 の機構観測は事後の記述的分析である。凍結した H1~H4b の達成条件は維持し、追加分析を新しい主判定にしない。群分けは全 98 対のほかに非 CT84 対、陰性対照 28 対、継続対象 70 対、両方を除く 56 対の 4 通りを併記する。これらは同じデータの部分集合であり、有意水準を調整した多重比較として設計していない。どの部分集計も、事前指定した達成条件の代わりに用いない。費用分解には §3 の較正で固定した単価を用い、CLI 報告額との残差を独立項目として保持する。再計算の定義とログ照合の範囲は付録 C に示す。

6.3 費用：20% 削減の達成条件は満たされなかった

指標	結果
相対費用差の中央値	+5.51%
相対差平均による符号反転検定	$p = 0.0013$
相対差中央値の BCa 95% 区間	$[-3.18, +10.12]\%$
C が安い対	44/98
総費用 B / C	58.35 / 62.91 USD
総費用の増加率	+7.82%

表 4: 全体の費用結果。総費用の比と対ごとの比の中央値は別の統計量である。

中央値は削減方向ではなく、95% 区間は 0 を跨ぐため、H1 の複合的な達成条件は満たされない。一方、区間の下端は -20% より上にあるため、分析上の仮定の下では、今回の集約中央値について 20% 以上の削減とは整合しない。これは「効果が全くない」「全ての仕様で削減できない」という結論ではない。図2に、値が広く分散し、C が安い比較も多数あることを示す。

C が安い対は 44/98 (44.9%) で、差の正負だけを用いた両側正確符号検定は $p = 0.3634$ だった。一方、相対差の平均は +8.37% で、平均を統計量とする符号反転検定は $p = 0.0013$ となる。頻度と差の大きさを含む量は別であり、後者の小さな p 値を「多くの対で C が高い」という強い証拠へ読み替えない。全体では高い対 54 件が安い対 44 件を上回るので、中央値の正符号はその順位構成と整合する。中央値が少数の極端な増加に引っ張られた、とは説明しない。

6.4 仕様別と CT 除外の感度分析

本節は前節と逆向きに見えるが、対象の統計量が異なる。中央値は順位で決まるので仕様構成に、平均は差の大きさで決まるので CT の大きな差に感度が高い。どちらも同じ 98 対の性質であり、一方を他方の誤りとししない。

ct_library では相対費用差中央値が +50.6% で、C が全 14 対で高かった。他の 6 仕様では相対差中央値が -7.9% から +12.1% に分布した。CT を除く副次集計 84 対では中央値 -1.4%、平均ベースの相対尺度の $p = 0.4950$ 、BCa 95% 区間 $[-5.10, +6.44]\%$ だった。C が安い 44 対は全てこの非 CT 群にあり、44/84 (52.4%) と僅かに多数になる（符号検定

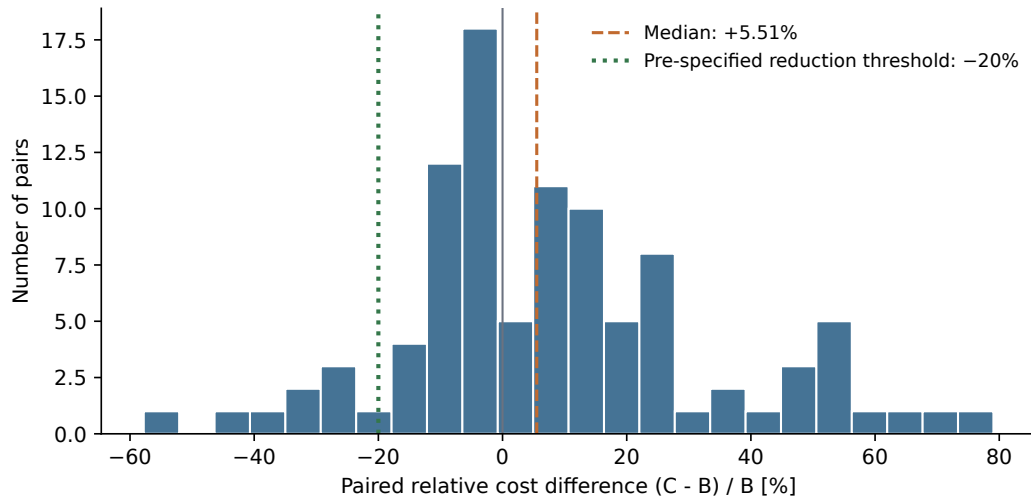


図 2: 全 98 対の相対費用差。破線は中央値、点線は 20% 削減の達成基準。各比較の変動と集約結果を区別する。

仕様	B 中央値	C 中央値	相対差中央値	C が安い対
s17_w_two_files	0.5547	0.5293	-7.9%	10/14
n_db_chain	0.4323	0.4467	-5.6%	10/14
s17_n_db_chain	0.4501	0.4790	-1.4%	8/14
s17_m_mixed	0.7451	0.7907	+5.0%	5/14
m_mixed	0.8054	0.8501	+5.2%	6/14
w_two_files	0.3609	0.3722	+12.1%	5/14
ct_library	0.6617	1.0255	+50.6%	0/14

表 5: 仕様別の費用 (USD) と対ごとの相対差。B/C の中央値同士から相対差中央値を再計算することはできない。各仕様 14 対。

$p = 0.7436$)。CT14 対を加えることで、正の差は 40 から 54 件へ増え、中央値の符号も変わる。平均相対差を全 98 対に対する加法的寄与に分けると、CT は +7.12 ポイント、非 CT は +1.25 ポイントである。CT は対の件数では 14/98 を占め、全体平均 +8.37% の大部分に寄与する。これは平均の分解であり、中央値 +5.51% の加法分解ではない。

さらに W2 仕様と CT の両方を除いた 56 対、すなわち直接競合ではない形状で継続が実際に起きた対では、中央値 -0.65%、BCa 95% 区間 $[-4.77, +6.86]\%$ 、C が安い対が 29/56 (符号検定 $p = 0.8939$) だった。区間は 0 を含み、-20% も +20% も含まない。ただし費用側には事前指定した同等性マージンがないため、これを「費用中立が示された」とは読まない。記述として、この範囲では大きな増減のどちらも観測されなかったということである。

全体の中央値の符号が仕様構成に敏感であることは言えるが、CT を除いた値を主要な結果へ差し替えない。CT は 1 仕様なので、形状一般の効果とその具体的な課題内容を分離できない。図3では全観測を仕様別に示す。

6.5 構造的陰性対照から見た変動

W 形状の w_two_files と s17_w_two_files は連結成分が全て単一タスクで、会話継続も、それによる追加の直列化も起きない。保存された C 側 56 呼び出しも全て resumed=false である。B/C は試したスケジューリングと継続の操作に関して一致するため、これら 28 対を構造

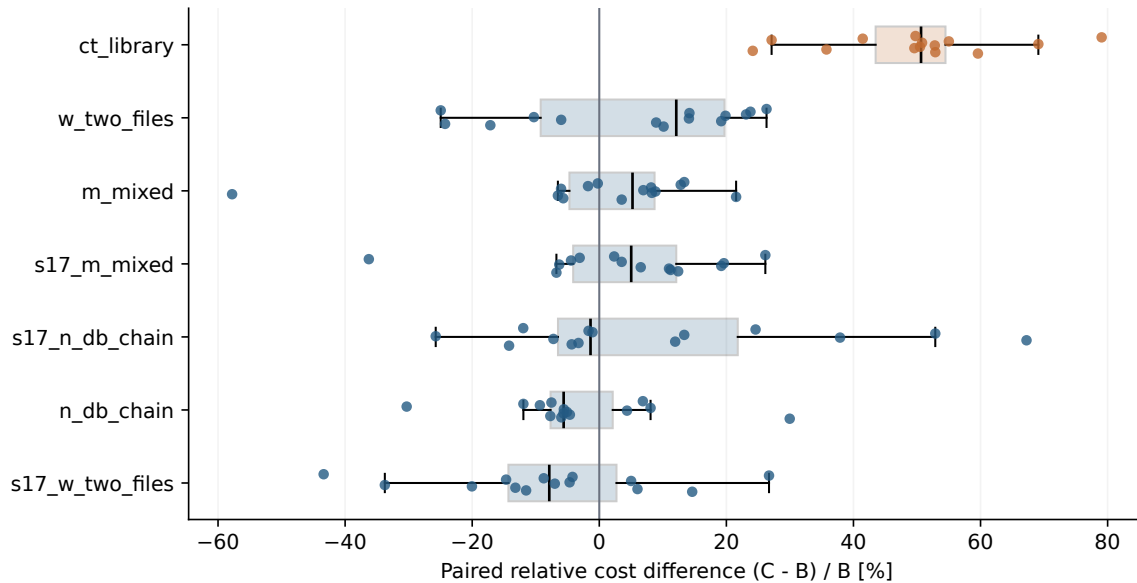


図 3: 仕様別の全観測と箱ひげ図。CT の 14 対は全て 0 より大きい。他仕様では正負の値が混在する。

的陰性対照として読む。両ランは別々に実行され、乱数的な生成、時間、nonce などまで同一にはしていない。

仕様	<i>n</i>	相対差中央値	BCa 95% 区間	USD 差中央値
s17_w_two_files	14	-7.9%	[-14.67, +0.40]%	-0.0376
w_two_files	14	+12.1%	[-10.29, +19.53]%	+0.0404
ct_library	14	+50.6%	[+41.46, +53.95]%	+0.3422

表 6: 陰性対照 2 仕様と CT。区間は各仕様の相対差中央値に対する事後の BCa 区間であり、仕様間の差の検定ではない。

s17_w_two_files の中央値は -7.9%、w_two_files は +12.1% である。全体の +5.51% はこの 2 点の間にあり、他の非 CT 仕様の中央値も同じ範囲に入る。その範囲の上にある仕様別中央値は CT の +50.6% だけである。ただし、この範囲は 2 つの点推定の最小・最大であって、測定系のノイズ上限ではない。W の個々の対は -43.4% から +26.7% まで広がり、CT の最小値 +24.2% と一部重なる。「CT だけがノイズを統計的に超えた」という判定を、この比較から導かない。

W をプールした 28 対の中央値は -4.44%、BCa 95% 区間は [-11.52, +10.12]%, C が安い対は 15/28 だった。単一の対照効果で他仕様を補正する根拠はなく、差し引き補正はしない。各 W 仕様は 14 対に限られ、B 費用中央値も 0.36 と 0.55 USD であるため、費用規模やタスク固有の変動を他仕様と共通とは仮定できない。図4に全観測と区間を併記し、小さな集約差と、観測した CT の一貫した大きな増加を区別する。

6.6 CT の料金項目とタスク別の費用分解

CT では 14 ランズつの報告総費用が B の 9.5434 USD から C の 14.2279 USD へ増え、差は 4.6845 USD (+49.09%) だった。以下はこの総額差を対象とし、対ごとの相対差中央値 +50.6% を分解したものではない。

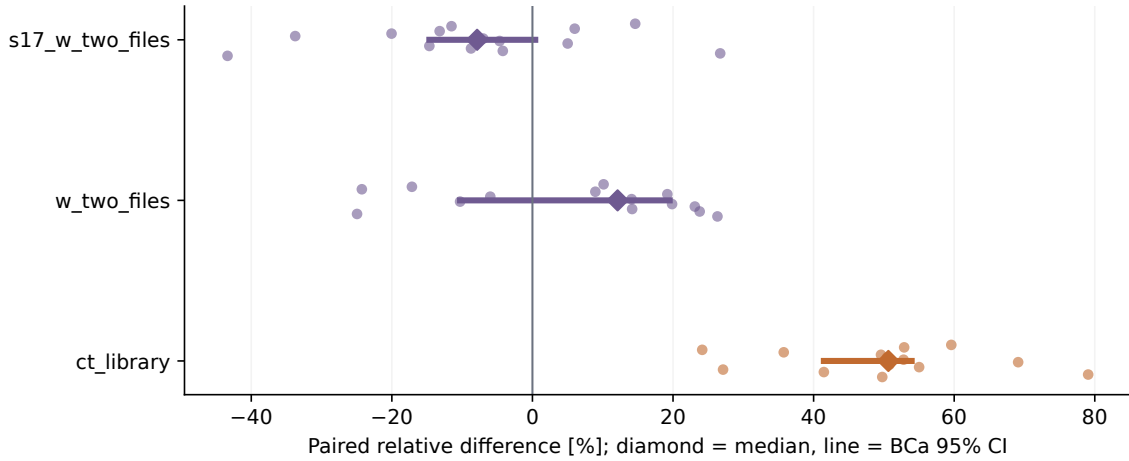


図 4: 陰性対照と CT。点は 14 対、菱形は中央値、横線はその BCa 95% 区間。対照 2 件の中央値を背景帯や有意性閾値として描かない。

トークン 4 項目を T_k 、既存校正の固定単価を p_k として、報告額との残差 ϵ を定義する。

$$c_{\text{reported}} = \sum_{k \in \{\text{in, out, read, write}\}} p_k T_k + \epsilon, \quad \Delta c_{\text{reported}} = \sum_k p_k \Delta T_k + \Delta \epsilon. \quad (7)$$

単価は 100 万トークン当たり通常入力 3、出力 15、cache read 0.30、cache write 3.75 USD である。これは校正コードの単価による標準化であり、本実験のモデル別請求単価を独立に検証したものではない。保存された usage と total_cost_usd は異なる集計項目で、モデル別内訳を復元できないため、残差をゼロに合わせ込まない。

項目	トークン差	B	C	差額
通常入力	-6	0.0032	0.0032	-0.0000
出力	+5,527	2.1744	2.2573	+0.0829
cache read	+6,434,057	5.9230	7.8532	+1.9302
cache write	+917,288	3.5554	6.9952	+3.4398
報告額との差 (残差)	—	-2.1126	-2.8810	-0.7684
報告費用	—	9.5434	14.2279	+4.6845

表 7: CT の 14 ラン対の加法分解。単位は USD。B 列・C 列はトークン 4 項目の固定単価換算で、最終行の残差だけは換算額ではなく報告額との差引項である。4 項目と残差の合計が CLI 報告額と一致する。通常入力差は -0.000018 USD。

固定単価での増分 5.4529 USD のうち、cache write が 3.4398 USD (63.1%)、cache read が 1.9302 USD (35.4%) を占める。報告費用との差の変化は -0.7684 USD であり、無視できる丸め誤差ではない。したがって上記の比率は固定単価で換算した増分に対するもので、実請求増加の内訳比率ではない。cache read には固定文脈、現在のタスク内履歴、継承履歴が混在し、トークン数だけで継承元を識別できない。

CT の cache read は 19,743,378 から 26,177,435 トークン (+32.6%)、cache write は 948,096 から 1,865,384 (+96.8%)、出力は 144,960 から 150,487 (+3.8%) へ増えた。CLI 報告ターン数の合計は 587 から 572 (-2.6%) であり、全体の費用増加をターン数の増加だけでは説明できない。記録されたターン詳細の行数は 551/530 と報告数に一致せず、ターン別入力集計にも一部不一致があるため、これらから完全な 1 ターン別請求を再構築しない。

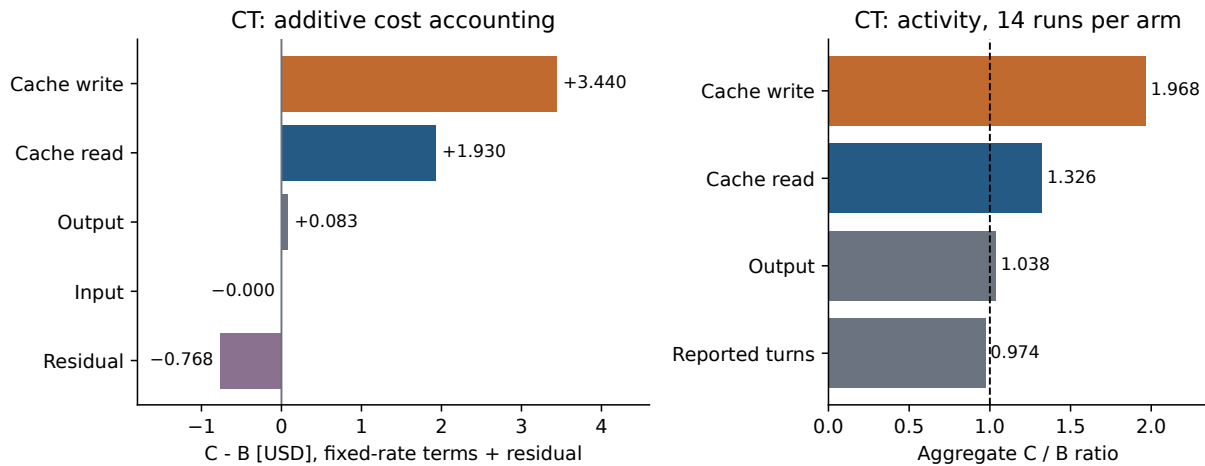


図 5: CT の固定単価による費用分解（左）と使用量・報告ターン数の比（右）。負の残差も表示し、換算額と CLI 報告額の違いを残す。

タスク	B 費用	C 費用	差額	ターン数 B/C	C で継続
CT-A	3.5933	4.9741	+1.3808	238/316	なし
CT-B	3.2771	4.5713	+1.2942	201/153	14/14
CT-C	2.6729	4.6824	+2.0095	148/103	14/14

表 8: CT のタスク別報告費用（14 反復合計、USD）と CLI 報告ターン数。CT-A は C でも新規セッションである。

タスク別では、C でも新規で始まる先頭の CT-A に 1.3808 USD、すなわち CT 総増加の 29.5% が現れた。CT-A のターン数は 238 から 316 へ増え、継続する後続 2 タスクでは合計 349 から 256 へ減った。よって、CT の全増加を「継承履歴を読む直接費用」だけに帰属させる説明は、会計段階でも支持されない。先頭タスクも実行順制約が変わった環境で動き、生成の変動もあるため、この差を特定原因へ割り当てない。 $\S 3$ の H_i 、 ρ 、 K'_j を代理量から埋めることはモデル計算であり、この料金項目別の観測から carry 式の各項を直接測定したことにはならない。

6.7 継続に伴う機構の観測

費用の増減がどの量の変化に対応するかを、CT の 3 タスクについて記述する。以下も事後分析であり、H1～H4b の判定を置き換ええない。

まず、セッションの再利用と使用モデルの構成が完全に対応していた。新規セッションの 434 件の呼び出しでは全てに補助モデル (claude-haiku-4-5) の使用が記録され、継続セッションの 84 件では 1 件も記録されなかった。call 記録 518 件に例外はなく、7 仕様のいずれでもこの対応は崩れない。したがって B 側と C 側で 1 ターンあたりの量を比べると、履歴の量だけでなくモデル構成の違いも含まれる。ログはモデル別の金額とトークン数を保存しないため、この 2 つを分離できない。表 9 のターン単価比は記述として示すが、これを継承履歴の処理費用とは読まない。

継続した 2 タスクでは、ターン数が減り出力がほぼ変わらない一方、最大文脈量が 44,044 から 64,482 (+46%)、38,408 から 73,340 (+91%) へ増えた。作業量の代理量が減っても費用が増えるという関係は、式(1)が扱う向きと一致する。ただし文脈量には固定文脈と現タ

タスク	C で継続	ターン差中央値	最大文脈 B→C	ターン単価比	費用相対差中央値
CT-A	なし	+5.5	43,290→52,637	1.04	+42.8%
CT-B	あり	-3.0	44,044→64,482	1.83	+37.0%
CT-C	あり	-3.0	38,408→73,340	2.52	+84.1%

表 9: CT のタスク別。ターン差と費用相対差は 14 反復の対応差の中央値、最大文脈は各アームの中央値。ターン単価比はモデル構成の違いを含む量であり、履歴処理の単価ではない。

スク内の履歴も含まれるので、増加分を継承履歴だけに割り当てない。

継続しない先頭の CT-A でも費用が増えた。14 反復の対応差の中央値は +42.8%、BCa 95% 区間 [+28.39, +49.17]% で、ターン数は 14/14 の反復で増え、最大文脈と出力も増えた。CT-A は B 側でも C 側でも新規セッションであり、この差を継続そのものでは説明できない。両アームで異なるのは実行順の制約であって、先頭タスクへ与える入力ではない。本稿はこれを未解決の観測として提示し、原因を特定しない。

式(1)に入れる継承量 $\hat{H}_i$ には、直前タスクの B 側較正での最大文脈を用いた (§3.4)。これに対し本実験で観測された文脈の増加は、CT-B で仮定量の 49%、CT-C で 81% にとどまった。仮定より小さい継承量しか観測されないことは、継続対象 5 仕様が全て過大予測だったこと (§6.9) と向きが一致する。ただし最大文脈は 1 ターンあたりの平均ではなく、継続しない CT-A でも文脈は増えており、較正は 1 反復で本実験は 14 反復である。したがってこれは候補となる説明であって、モデルの検証ではない。

最後に、報告費用を固定単価換算で割った比は、7 仕様 2 アームの 14 点で 0.78 から 0.85 に収まった（全体で B が 0.804、C が 0.820）。残差は加法的な誤差ではなく、ほぼ一定の比率として働いている。安価な補助モデルが換算に含まれないことは、この比が 1 を下回る向きと整合する。比が 2 アームで近いことは、§6.6 の項目シェアが残差によって大きく組み替えられる状況ではないことを示す。ただし残差の項目別内訳は観測できないので、シェアが保存されることの証明ではない。単価を実測総額へ合わせ直すことはしない。

6.8 時間と受入テスト

時間の中央値は B が 280.85 秒、C が 292.80 秒で、比は 1.0425 だった。対応を保ったブートストラップによる片側 90% 上限 1.1335 は 1.15 を下回り、指定した時間の非劣性基準を満たした。これは時間が同じという証明ではなく、採用した 15% の許容差についての判断である。

受入全通過は B が 98/98、C が 97/98 で、差は -1.02 ポイントだった。通過率差の片側 90%BCa 下限は -5.10 ポイントで、-10 ポイントの基準を満たす。この許容差は比較的広く、1 件しか不一致がないデータから品質全般の同等性を保証しない。補助確認として、B 成功・C 失敗の確率の片側 90%Clopper-Pearson 上限を反転した保守的な通過率差下限は -3.91 ポイントだった（付録C）。

6.9 モデル予測の一致と不一致

仕様 s の実測比は $r_s = \sum_i C_{si} / \sum_i B_{si}$ であり、表5の相対差中央値とは異なる。事前指定の全 7 仕様での評価では、相対誤差 25% 以内が 6/7、全仕様の方向一致は未達であり、H4b は成立しない。ただし W2 仕様の予測値 1.0000 は、継続が発生しない構造から決まる。この 2 件を分ける事後の補助集計では、実質的な継続対象 5 仕様のうち 4 件が 25% 以内で、外れるの

仕様	実測 r_s	予測 $\hat{r}_s$	相対誤差	25% 以内
ct_library	1.4909	1.5575	-4.3%	はい
m_mixed	1.0060	1.1806	-14.8%	はい
n_db_chain	0.9571	1.5251	-37.2%	いいえ
s17_m_mixed	1.0205	1.2987	-21.4%	はい
s17_n_db_chain	1.0736	1.3730	-21.8%	はい
s17_w_two_files	0.9021	1.0000	-9.8%	はい
w_two_files	1.0334	1.0000	+3.3%	はい

表 10: 固定した予測との比較。相対誤差は $(r_s - \hat{r}_s)/\hat{r}_s$ 。等値の予測 1.0000 は増加の予測ではない。

は n_db_chain (-37.2%) だった。

継続対象 5 仕様では予測が実測を全て上回った。全 7 仕様での過大予測 6/7 より、モデルが継続を扱う部分の偏りを直接表す記述である。ただし同じモデル式と較正手順を共有する 5 点なので、独立した公平な符号試行として「偶然では説明しにくい」とは評価しない。この偏りは、較正 1 反復、 $\rho = 1$ 固定、探索区間の代理量、累積履歴の近似に加え、固定単価と CLI 報告額の不一致を点検する理由になる。原因や補正可能性を確立したわけではなく、現データに合わせた予測の再較正は行っていない。

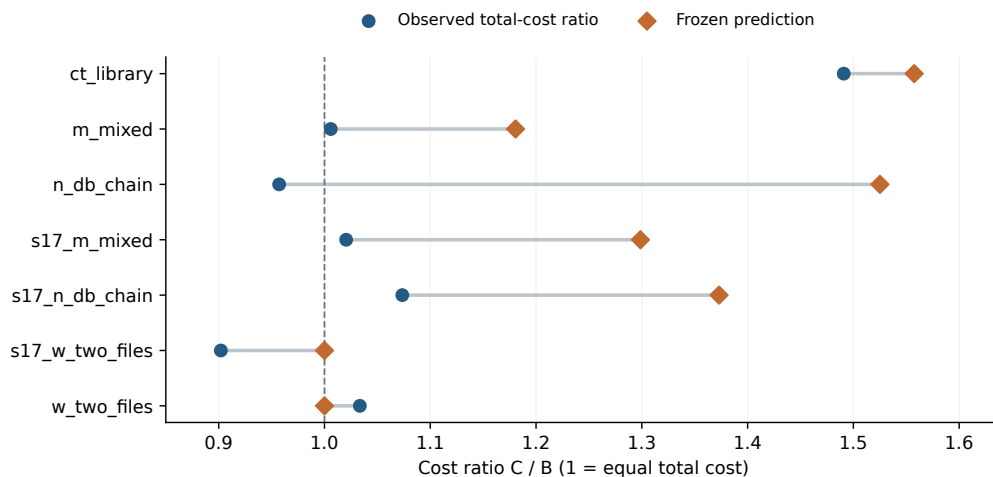


図 6: 予測と実測。継続対象 5 仕様は全て過大予測。下 2 件の W は構造上 1.0000 を返す対照であり、実質予測の補助集計から分ける。破線の 1.0 は両方式の総費用が等しい境界。

6.10 中心主張と追加分析

「時間と品質を許容範囲に保ち、費用を 20% 以上削減する」という中心主張は、全ゲートを満たした場合だけ成立する。費用の条件を満たさず、中心主張は支持されない。時間と受入の非劣性のみを、マージン付きで報告する。

改稿時の追加分析として、7 仕様それぞれの 14 対を仕様内で復元抽出するブートストラップを行った。プールした中央値の percentile 95% 区間は $[-2.17, +8.84]\%$ だった。仕様ごとの反復構成を保っても、この分析の範囲では 20% 削減と整合しない。これは固定 7 仕様に条件づけた追加分析であり、リポジトリ母集団の区間ではない。10 分以上離れた 11 対を除く 87 対でも中央値は $+6.0\%$ 、BCa 区間は $[-3.27, +8.95]\%$ で、費用削減の達成条件を満たさなかった。

7 支持されなかった見立て

本節は、測定の過程で採らないことにした説明と、本稿の成果に含めない事柄を挙げる。新しい主張は置かない。機構についての観測は §6.7 に示した。

7.1 キャッシュ共有を単純な二択で捉えない

当初は、別セッションで同じ入力を送れば共有できるか、全く共有できないかという二択で考えた。実際には固定文脈が共有され、新規から新規と、継続後の新規とで観測が異なった。したがって「新規セッションにはキャッシュ利得がない」という一般的な説明は採らない。比較実験ではその条件依存性を踏まえ、nonce と実行順の記録を用いた。

7.2 履歴の増大と書き直しの関係

M6 では、継承履歴が増えても後続ターンの cache write が新規追加分で一定の系列を得た。履歴が長いことと、毎回その全量を書き直すことは同じではない。M7 は追加プロンプト量だけで初回書き直しを説明する見立ても支持しなかった。一方、履歴を cache read で繰り返し処理する費用は残る。機構の一部を観測できたことから、仕事全体の正味利益までを推定しない。

7.3 予算付きレーンを成果としない

履歴が予算を超えたら新規に切り替える C' も検討したが、パイロットでは継続の費用増加が観測され、独立した本実験アームとして採用しなかった。少数の観測から厳密な損益分岐 H^* を同定した、あるいは C' が一般に B と同じ動作になると証明した、とは言わない。適応スケジューラ D、助言注入、他方式の再帰的協調も測定していない。「改善候補を考えた」ことを「改善を実証した」ことへ置き換えない。

8 妥当性と適用範囲の限界

8.1 対象の狭さと仕様への依存

単一言語・単一リポジトリ・単一モデル識別子・単一 CLI 版であり、仕様の作成・選定には著者の判断がある。7 仕様の反復は、7 つの独立したシステムへの再現実験ではない。特に CT 仕様は 1 件なので、同じグラフ形状の他の課題に同じ費用差が出るとは言えない。実運用における CT の出現頻度も測っていない。

8.2 複合介入と観測できない状態

セッション継続と実行順制約を同時に変えているため、費用差の因果的な分離はできない。料金項目別の換算とタスク別の分解は可能だが、各原因の効果とは異なる。初期 cache read との対応は、内部キャッシュ機構の因果的な同定ではない。nonce で全共有を遮断した保証はなく、固定文脈やプロバイダ状態は残る。プローブと本実験の CLI 版も違うため、小規模プローブの結果を本実験の全境界へそのまま移さない。

8.3 順序・運用変更と統計上の仮定

49 対ずつの先行方式の均衡は確認したが、アーム順は反復に依存し、時間変動との独立性を保証しない。A-5/A-6 などの変更、再実行、利用上限への対応がある。感度分析が集約結論を変えなくても、あらゆる交絡を排除したとは言えない。符号反転検定とブートストラップは、対の独立性や交換可能性の仮定に依存する。近接した反復のサービス状態に相関があれば、不確実性が過小評価されうる。

8.4 モデル近似と較正の量

較正は各仕様 1 反復なので、探索量や履歴量の分散を十分に推定できない。最初の編集以前を探索とする定義では、編集後の再探索を作業側に分類する。 $\hat{S}$ は最小観測文脈であり、固定システム文脈そのものの測定値ではない。継続後の出力量、ツール行動、履歴の累積を十分に予測していない。探索／作業の境界をずらす感度分析は実施しておらず、固定予測を今回の結果に合わせて更新していない。

8.5 品質の範囲と少数の不一致

受入テストは機械的だが、全ての品質属性を表さない。10 ポイントの許容差は運用判断で、品質が同じであることを示す基準ではない。二値の不一致が少ないため、補正後の BCa でも小標本・離散分布の限界がある。正確二項上限を用いる補助解析も、独立で共通の不利益確率を持つという仮定の下での確認である。

8.6 装置を再配布しないことによる再現の非対称

本研究の再現可能性は分析側と実験側で非対称である。196 ランの計測データは同梱するため、集計・判定・図表の再実行は本パッケージだけで完結する。一方、実験を回した実行フレームワークと、実験された対象リポジトリはいずれもライセンスが設定されておらず、本研究では再配布しないと決めた。公開物に含まれるのは実験手順と仕様の記述であって、装置そのものではない。

したがって第三者は、本稿の数値を独立に再取得して検証することができない。検証できるのは、同梱データから同じ結論が導かれるかまでである。判定規則を機械適用した点と事前指定文書の保存は、分析者の自由度を封じるものであり、データ生成そのものの独立検証の代わりにはならない。これは変更不能な商用 API を私有リポジトリ上で測る設計に共通する制約だが、共通であることは制約を消さないため、主張の範囲をここに留める。

8.7 非盲検と公開前の証拠

研究の設計者はパイロットの方向を見ており、C' の着想も観測後である。B 側だけで較正したことは C/B 比への直接の適合を避けるが、完全な盲検ではない。またローカル Git の日時は改変可能であり、それだけで独立した事前登録時刻を証明しない。本稿は保存された事前指定文書と変更履歴を開示するが、外部の登録・公開を完了したとは述べてない。

9 結論

商用コーディングエージェント CLI の測定と 98 対の比較で、会話継続による 20% 費用削減の達成条件は満たされなかった。事後分析では、継続がない W2 仕様にも -7.9% と $+12.1\%$ の中央値が現れ、全体 $+5.51\%$ を一般的な継続効果として解釈することに注意を促した。この 2 点の範囲はノイズ上限ではないが、CT 仕様だけがその範囲を上回る中央値 $+50.6\%$ を持ち、全 14 対で高かった。非 CT では 44/84 対で C が安く、全体平均の増加は CT の大きな差に集中した。

CT では総ターン数が減る一方、cache write と read が増え、固定単価換算では write が増分の最大項目だった。報告額との残差と、継続前の先頭タスクに現れた増加を含めて示すことで、料金項目の増減と継承履歴の因果効果を区別した。継続対象 5 仕様でのモデル過大予測は、履歴モデルと単価・計装を再点検する具体的な根拠である。

次に必要なのは、異なる課題内容を持つ CT 仕様を複数追加し、同じ実行順で継続の有無を比較する実験である。対照仕様も費用規模を揃えて増やし、モデル別費用と完全なターン計測を保存することで、今回の異質性と残差を独立に検証できる。現段階の所見は観測した仕様と CLI に限られ、CT 形状一般や未評価の適応方式の優位性までは確立しない。

A 分析訂正の履歴

v0.2 の H1 では宣言済み相対差への入力尺度の訂正を、H3 では中央値から通過率差に対応する平均への訂正を行った。旧 H3 の下限 0.00 ポイントは 97/98 対の差が 0 だったことによる中央値の集中であり、通過率差の区間ではなかった。旧コード・JSON は保持し、一時領域で再実行して完全一致を確認した上で別コードに訂正結果を保存した。H4b の旧二値規則 ($r > 1$) $\neq$ ($\hat{r} > 1$) の不一致は `n_db_chain` と `w_two_files`、増加・等値・減少の 3 値では `s17_w_two_files` も不一致となる。いずれも全一致条件を満たさない。旧稿の CT 除外 $p \simeq 0.90$ は USD 尺度であり、本文の相対尺度 $p = 0.4950$ とは異なる。

項目	保存した旧結果・旧説明	本稿の扱い
H1 の尺度	USD 差の $p = 0.0047$ と区間	分析前文書の相対差へ訂正。 $p = 0.0013$ 、区間 $[-3.18, +10.12]\%$ 。旧 USD 中央値区間は $[-0.0173, +0.0581]$
H3 の統計量	中央値の下限 0.00 ポイント	通過率差の平均へ訂正。下限 -5.10 ポイント。基準通過は維持
H4b の方向	等値を含む二値判定を「2 つの値下がり予測ミス」と説明	旧コードの不一致集合と 3 値記述を分ける。いずれも全一致せず
予測の偏り	7/7 で過大予測	表と計算に一致する 6/7 へ訂正
TTL の記述	約 10.2 分まで warm と断定	612.1 秒・916.9 秒の保存分類は ambiguous。寿命の確定値を撤回
失敗時の継続追加の区間	必ずセッション破棄 未掲載	実装にその一律保証がないため撤回 仕様内復元抽出の percentile 区間、受入の保守的二項下限を事後分析として掲載

表 11: 2026 年 9 月 4 日の統合時の訂正。原データと判定閾値は変更していない。

旧版は `paper/revision/original/` に保存し、ファイルごとの SHA-256 を `manifest.json` に記録した。凍結結果は引き続き `analysis/e2_results_main.json`、訂正結果は `paper/`

revision/results.json である。主要結論が変わらないことと、計算・説明に誤りがなかったことは同じではない。本稿は誤りを訂正し、旧結果の再現可能性を保つ。

B 機構測定の記録と未確定事項

probes/resume_probe_results.jsonl には初期試行と有効な反復をともに保存している。M1 の一意接頭辞付き 3 反復は、2026 年 7 月 31 日 02:09 台の init/resume 対である。その前の共有接頭辞による試行と混ぜない。M4 の 3 対では cache write/read 内訳が一致する一方、出力を含む総費用比には 1.082 の例がある。したがって「総費用が全て完全一致」ではなく、キャッシュ内訳の一致として報告する。

長間隔プローブは 612.1 秒と 916.9 秒の 2 件で、保存判定はどちらも ambiguous である。612.1 秒の cache write/read は 29,572/15,912、916.9 秒では 29,417/15,912 だった。固定文脈の一部が残る状態と履歴全量の再利用を区別でき、厳密な保持時間の推定値としては扱わない。初期の約 6.4 分後の読み取り観測もあるが、同一セッションの継続測定で寿命更新の影響を含む。warm の機構チェックを満たすことから、H4a の長間隔・切替の全条件まで合格したとはしない。

m6_results.jsonl と m7_results.jsonl は追加量と初期 cache read の所見を支える。M7 の 12 測定と、M6 を含めた初回状態の例を混在した表を同じ標本数として扱わない。提供側のブレイクポイント配置などの内部原因は、このログだけから確定できない。

C 追加分析の定義

固定 7 仕様の層別ブートストラップでは、各仕様から 14 対を復元抽出し、合計 98 対の相対差中央値を計算する。これを 10,000 回繰り返し、線形補間した 2.5・97.5 百分位を用いた。仕様そのものを無作為に採取した設計ではないので、仕様数を増やす母集団推論とは区別する。

受入について、 $q = P(B\text{成功}, C\text{失敗})$ 、 $g = P(B\text{失敗}, C\text{成功})$ なら、通過率差は $g - q \geq -q$ である。観測した不利益不一致は 1/98 なので、二項分布の片側 90% 正確上限 U_q [3] を用いると、 $-U_q$ が保守的な通過率差下限となる。 $\sum_{j=0}^1 \binom{98}{j} U_q^j (1 - U_q)^{98-j} = 0.10$ を解き、下限 -3.91 ポイントを得た。各対に共通の不利益確率と独立性を仮定する補助確認であり、仕様別の受入品質を保証しない。

C.1 v0.3 の追加分析と計装の照合

陰性対照と仕様別区間は既存の BCa 関数を用い、再標本数 10,000、種 20260802 を維持する。正確符号検定は同値を除いた n 対の正負数を n_+, n_- として、 $\min\{1, 2 \sum_{j=0}^{\min(n_+, n_-)} \binom{n}{j} 2^{-n}\}$ を用いた。今回同値はない。独立した対で正負確率が等しい帰無モデルに対する補助検定であり、アーム順の無作為化を主張しない。継続対象だけの 70 対も保存 JSON に収録し、相対差中央値 +6.90%、BCa95% 区間 $[-1.42, +12.17]\%$ だった。

calls.jsonl は追記履歴であり、旧実行器は最終プロセスの usage を run.json へ書く。全 196 ラン中 195 ランは呼び出し履歴の末尾部分の 5 集計項目（4 トークン数と費用）が最終 usage に一致した。7 ランでは余分な先行履歴があり、m_mixed/B/rep5 は単純な末尾合計では一致しなかった。このランも主要費用と全仕様のトークン集計から除外せず、run.json を用いる。CT 全 28 ランは各 3 呼び出しの全行が 5 項目とも一致し、タスク別分解はこの検証済み範囲で行った。

補助モデルの使用記録は全ランで確認した。call 記録 518 件のうち、新規セッションの 434 件は全てが代表モデルに加えて補助モデル名を持ち、継続セッションの 84 件は 1 件も持たない。7 仕様のいずれでも例外はなく、CT での内訳は B 42/42、C 14/42 である。詳細なモデル別 `modelUsage` の数値は保存していないため、補助モデルが費用と所要ターンへ与えた寄与は測れない。このため 1 ターンあたりの量をアーム間で比べる記述は、履歴量とモデル構成の両方を含む。

CT のタスク別対応差は、同じ反復番号の B 側と C 側の呼び出しを対応付けて算出した。最大文脈量は各アーム 14 反復の中央値である。継承量の照合には較正時（アーム B、1 反復）の最大文脈を用いており、本実験の反復数と一致しないので、比は概算の対比として扱う。報告費用と固定単価換算の比は仕様別・アーム別に算出し、残差の項目別内訳は復元していない。CT の B 側ではターン行の `cache read` 合計が `result` 集計より 632,352 トークン、`write` が 26,734 トークン多い。C 側は一致する。`output_tokens_partial` は開始時点の未確定値なので費用集計に使わない。CLI 内部の集計範囲、補助処理、料金体系をこの保存形式から一意に復元できず、残差の原因を単価差だけへ帰属させない。料金項目の増減、モデル別請求、継承履歴だけの費用を分けるための限界である。

D 再現方法と成果物

統合本文の正本は `paper/main.tex` である。数値・表・図は訂正結果 JSON と同じデータセットから生成する。

生成スクリプトは `paper/render_assets.py` である。追加結果は `paper/revision/v0.3/results.json` に保存する。原稿と図表を作り直す手順は次のとおり。

```
python3 analysis/revision_audit.py
python3 analysis/supplement_v03.py
.venv-figs/bin/python paper/render_assets.py
bash paper/build.sh
```

分析訂正は標準 Python ライブラリのみ、作図は NumPy/Matplotlib、PDF 生成は XeLaTeX 互換の Tectonic と日本語 Noto CJK フォントを用いる。ビルドスクリプトは取得済み Tectonic または PATH 上の実行ファイルを使う。未取得の TeX 資材には初回のネットワークアクセスが必要となる。`paper/BUILD.md` に環境とファイルの役割を記す。

分析は保存データだけで再実行できるが、本実験の再実行には商用サービスへのアクセス、実行フレームワーク、対象リポジトリの固定版が必要である。本原稿のソース一式だけで 196 ランを再取得できるとは主張しない。著者表示・所属・連絡先、公開リポジトリ URL、DOI、投稿先識別子は未確定のため作成していない。ソースと PDF をローカルに作成したことを、外部公開の完了と同一視しない。

参考文献

- [1] ccusage contributors. Cost modes. ccusage documentation, 2026. Accessed 2026-09-05. <https://ccusage.com/guide/cost-modes>.
- [2] ccusage contributors. Session usage. ccusage documentation, 2026. Accessed 2026-09-05. <https://ccusage.com/guide/session-reports>.
- [3] C. J. Clopper and E. S. Pearson. The use of confidence or fiducial limits illustrated in the case of the binomial. *Biometrika*, 26(4):404–413, 1934. <https://doi.org/10.1093/biomet/26.4.404>.
- [4] Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman & Hall, 1993. <https://doi.org/10.1007/978-1-4899-4541-9>.
- [5] Chaofan Lin, Zhenhua Han, Chengruidong Zhang, Yuqing Yang, Fan Yang, Chen Chen, and Lili Qiu. Parrot: Efficient serving of LLM-based applications with semantic variable. arXiv:2405.19888, version 1, 2024. <https://arxiv.org/abs/2405.19888v1>.
- [6] llm-d contributors. Precise prefix cache routing. llm-d documentation, 2026. Accessed 2026-09-05. <https://llm-d.ai/docs/well-lit-paths/foundations/precise-prefix-cache-routing>.
- [7] Elias Lumer, Faheem Nizar, Akshaya Jangiti, Kevin Frank, Anmol Gulati, Mandar Phadate, and Vamse Kumar Subbiah. Don’t break the cache: An evaluation of prompt caching for long-horizon agentic tasks. arXiv:2601.06007, version 2, 2026. <https://arxiv.org/abs/2601.06007v2>.
- [8] Belinda Phipson and Gordon K. Smyth. Permutation p -values should never be zero: Calculating exact p -values when permutations are randomly drawn. *Statistical Applications in Genetics and Molecular Biology*, 9(1):Article 39, 2010. <https://doi.org/10.2202/1544-6115.1585>.
- [9] Rakibul Hasan Rajib, Mengxin Zheng, and Qian Lou. AgentServeSim: A hardware-aware simulator for multi-turn LLM agent serving. arXiv:2606.09613, version 1, 2026. <https://arxiv.org/abs/2606.09613v1>.
- [10] Ray contributors. Prefix-aware routing. Ray documentation, 2026. Accessed 2026-09-05. <https://docs.ray.io/en/latest/serve/llm/user-guides/prefix-aware-routing.html>.
- [11] Cosmo Santoni. Contextual memory virtualisation: DAG-based state management and structurally lossless trimming for LLM agents. arXiv:2602.22402, version 1, 2026. <https://arxiv.org/abs/2602.22402v1>.
- [12] taekim34. Session resume (-continue) invalidates entire prompt cache, causes massive rate limit consumption. anthropics/claude-code, GitHub issue 42338, 2026-04-02, 2026. Practitioner report; accessed 2026-09-05. <https://github.com/anthropics/claude-code/issues/42338>.

- [13] Michelle Vaccaro. Preregistration for experiments with AI agents. arXiv:2606.11217, version 1, 2026. <https://arxiv.org/abs/2606.11217v1>.
- [14] Xu Yang, Lunyiu Nie, Ethan Chandra, Stanislav Gannutin, Fangru Lin, and Swarat Chaudhuri. When parallelism pays off: Cohesion-aware task partitioning for multi-agent coding. arXiv:2606.00953, version 1, 2026. <https://arxiv.org/abs/2606.00953v1>.
- [15] Ying Yuan, Pengfei Zuo, Bo Wang, Zhangyu Chen, Zhipeng Tan, and Zhou Yu. DualMap: Enabling both cache affinity and load balancing for distributed LLM serving. arXiv:2602.06502, version 1, 2026. <https://arxiv.org/abs/2602.06502v1>.
- [16] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient execution of structured language model programs. arXiv:2312.07104, version 2, 2024. <https://arxiv.org/abs/2312.07104v2>.